

Práctica 1 Simulación de un ambiente base e implementación de un robot móvil

,Arturo Reyna Reyes, Alexis Eduardo Bautista Rodriguez ,Humberto Guzmán Garduño

A01654610@itesm.mx, A01651493@itesm.mx , A0133748@itesm.mx,

Campus Ciudad de México

Engineering and Architecture Division

Mechatronics Engineering Department

CS95003.1 Taller de robótica aplicada

Introducción

Cada día los robots se están volviendo una parte cotidiana de nuestras vida, teniéndose en nuestros trabajos y hogares. Anteriormente para poder conocer el comportamiento y funcionalidad de un robot se debía de construir para poder probarlo, pero esto era un método ineficiente y costoso. actualmente tenemos diferentes herramientas para poder conocer el comportamiento y poder programar un robot sin necesidad de invertir recursos en pruebas físicas. Una de estas herramientas son los simuladores que existen actualmente como Matlab, Webots, SolidWorks, etc.

En estos simuladores se incorpora la parte de programación y de controladores para los movimientos del robot a crear. Se pueden aplicar diferentes lenguajes de programación como C++, C#, Java, Python y cualquier lenguaje que maneja orientado a objetos.

En esta práctica se utilizará el simulador Webots y el lenguaje de programación Python.

Objetivos:

Desarrollar un robot dentro de un ambiente controlado y programable en software WEBOTS.

Desarrollo

Abrimos el software webots y habilitamos el ambiente de trabajo..

Para habilitar un ambiente de trabajo para nuestro robot debemos de habilitar:

- TextureBackground
- TextureBackgroundLight
- Floor

Texture back ground es el fondo de la simulación. Texture Back ground Light es la ubicación de la iluminación y Floor el piso donde se realizara la simulación.



Figura 1. Ambiente de webots.

Agregamos el cuerpo de nuestro robot como un sólido.

Al sólido le agregamos dos objetos conocidos como *Children*. Estos los escogemos como los eslabones. Estos eslabones tendrán un punto final y una opción de mecanismo. El mecanismo debe programarse como motores rotacionales y los puntos finales los nombramos como *wheel1* y *wheel2*. En este punto agregamos dos sólidos los cuales serán las ruedas y debemos darles el aspecto o *Shapes* circulares para que sean las ruedas de nuestro robot.

Es importante agregar a cada uno de nuestros objetos creados la opción de *Physics*. Esto para que se pueda crear una buena simulación con el entorno creado.

Además incorporamos una tercera rueda al final para que haga la función de una rueda esférica. Esta no debe de controlarse ya que solo nos ayuda como un punto de apoyo para mejorar el control de nuestro robot.

Es importante recalcar que el robot se compone de 4 elementos: la base, las dos ruedas y la esfera que sirve de apoyo. Solamente las ruedas cuentan con el elemento de *Physics*, debido a que son las que interactúan con el entorno. Los otros elementos al ser de apoyo, pueden mantenerse sin ese parámetro y de esta forma se evitan complicaciones,

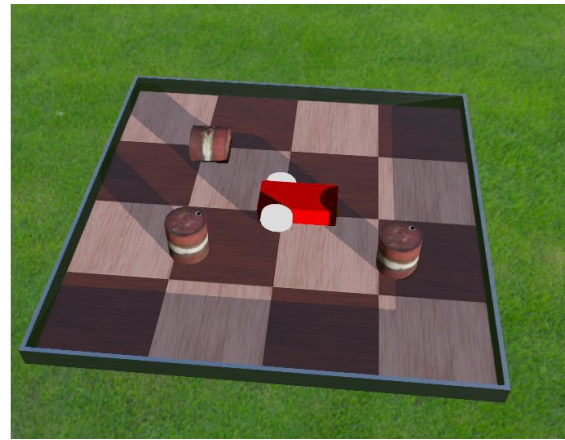


Figura 2. Diagrama de robot.

Importamos el controlador para empezar la programación

Aplicaremos un controlador PID para el control del movimiento del robot. Este controlador se aplica para tener un mejor control del movimiento y la velocidad del robot ya que sin este el robot puede avanzar sin control alguno.

Para el funcionamiento de este código importamos las librerías de *Keyboard*, *motor* y *PositionSensor*. Estos son para la lectura de las instrucciones que se mandarán del teclado, el control de los motores y el sensor de posición para el controlador PID.

Para el controlador PID escogimos los valores de sus coeficientes como:

$P=10$

$I=1$

$D=1$

Con esto podemos controlar la velocidad y posición utilizando los datos que tenemos del sensor de posición instalado en nuestro robot.

Los valores de los coeficientes fueron aplicados a base prueba y error, debido a que no se podían realizar pruebas para obtenerlos por alguno de los métodos conocidos.

Resultados

Se decidió dividir esta práctica en 3 puntos importantes:

- World
- Control por teclado
- Control PID

De esta forma poder evaluar los resultados de cada etapa.

En el caso de la creación del World se incluyeron, la edición de Floor y Textures. Eso se logró sin contratiempos. También en esta etapa se incluyó la creación del robot. La solicitud era que fuera de mínimo dos ruedas. Aunque eran elementos sencillos se tuvo un poco de problema en entender la forma en que se relacionaban los elementos entre ellos, ya que muchos son dependientes de otros, por lo que el orden en el que deben estar guardados debe ser específico. Sin embargo, ya que se logró entender la lógica detrás, se pudo realizar sin ningún problema.

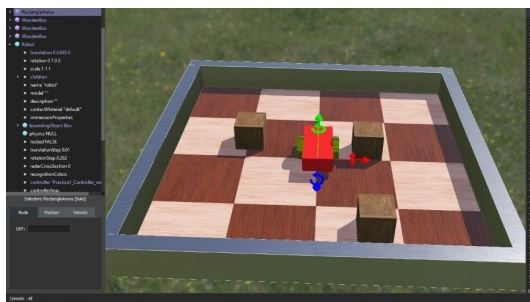


Figura 3. World con Robot

Teniendo la primer etapa resuelta, se procedió a realizar el código, el cual está conformado del control por teclado y el control PID. Aunque conocíamos como es un controlador PID y tenemos experiencia programando en Python, esta etapa nos resultó complicada debido a que se debieron realizar búsquedas para encontrar

la forma correcta de hacer el código. Se debieron de importar librerías relacionadas al control por teclado así como realizar varias pruebas en el simulador. Uno de los problemas que llegamos a obtener fue que el nombre de los motores debía ser el mismo que se había asignado en el código. Por esto, obtuvimos varias veces errores. También llegamos a obtener errores de sintaxis de código.

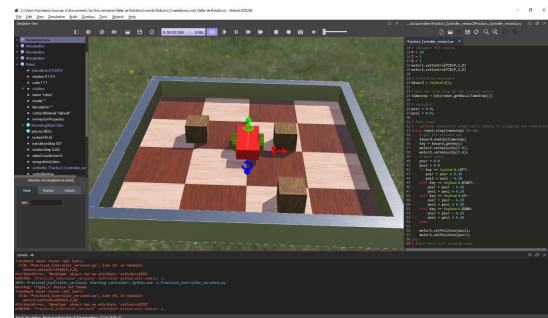


Figura 4. Interfaz Webots con error

También, para revisar errores de sintaxis se usó la plataforma Spyder, esto para hacer un mejor debugueo del código.

Figura 5. Interfaz Spyder

Mientras se corregía el código, se decidió incluir una tercera Joint, siendo la esfera del frente. Esto con el fin de ayudar en el movimiento del robot.

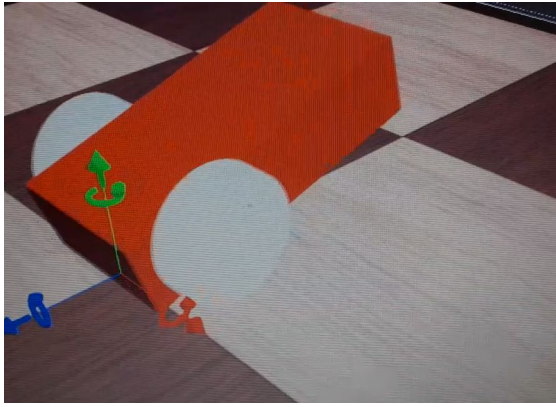


Figura 6. Robot en Movimiento

Finalmente, pudimos lograr implementar ambos controles en el World, logrando así, el objetivo de la práctica.

REFERENCIAS

- Webots User Guide. (s. f).
<https://cyberbotics.com/doc/guide/tutorial-1-your-first-simulation-in-webots>. Recuperado 11 de septiembre de 2020, de
<https://cyberbotics.com/doc/guide/tutorial-1-your-first-simulation-in-webots>
- Soft illusion. (2020, 11 marzo). Teleoperation with keyboard on Custom robot | Webots Simulator | [Tutorial 9] [Vídeo]. YouTube.
https://www.youtube.com/watch?v=wqXbrKxkDtM&list=PLt69C9MnPchlWEV5AEhfT2HajIE2SJ55V&index=9&ab_channel=Softillusion

Anexos:

Código python:

```
from controller import Robot
from controller import Motor
from controller import Keyboard
from controller import PositionSensor
# Then we create the Robot instance.
robot = Robot()
keyboard = Keyboard()
# Next we get the time step of the current world.
timestep = int(robot.getBasicTimeStep())
# We assign the timestep to the keyboard function
keyboard.enable(timestep)
# Next we initialize the motors
wheels = []
wheelsNames = ['m1', 'm2']
for name in wheelsNames:
    wheels.append(robot.getMotor(name))
# And the position sensors
ds = []
dsNames = ['sens1', 'sens2']
for i in range(2):
    ds.append(robot.getPositionSensor(dsNames[i]))
    ds[i].enable(timestep)
# We assign PID controller for motors
p = 10
i = 1
d = 1

# Then we initialize speed and position variables
error = 0
previous_error = 0
errorA = 0
error_integral = 0
error_derivativo = 0
speed = 0
posr = 0.0
posl = 0.0
a = 0.0
# Main loop:
```

```

# - perform simulation steps until Webots
is stopping the controller
while robot.step(timestep) != -1:
    # First we get the pressed key
    key = keyboard.getKey()
    # We determine the movement
    depending on the key
    if key == Keyboard.DOWN:
        posl = posl + 3.28
        posr = posr + 3.28
        print("Atras")
    elif key == Keyboard.UP:
        posl = posl - 3.28
        posr = posr - 3.28
        print("arriba")
    elif key == Keyboard.LEFT:
        posl = posl + 3.28
        posr = posr - 3.28
        print("Izquierda")
    elif key == Keyboard.RIGHT:
        posl = posl - 3.28
        posr = posr + 3.28
        print("derecha")
    # Next we use a PID controller to
    determine velocity
    previous_error = error;
    error = wheels[0].getTargetPosition()-
ds[0].getValue()
    error_integral += error * timestep
    error_derivative = (previous_error -
error) / timestep
    speed = p * error + d * error_derivative
+ i * error_integral ;
    if speed < 0:
        speed = -1.0 * speed
    if speed >= 10:
        speed = 10
    # We assign the movement to the
    motors
    wheels[1].setPosition(posr)
    wheels[1].setVelocity(speed)
    wheels[0].setPosition(posl)
    wheels[0].setVelocity(speed)
    pass

```